Mini Project Report

on

WanderLust

Bachelor of Technology

In

Computer Science and Engineering

By

1. Naman Sharma

Roll No.: 2406950100108

2. Prerak

Roll No.: 2406950100127

Under the Supervision of

Tanu Verma

Assistant Professor

Department of Computer Science and Engineering



SHRI RAM GROUP OF COLLEGES
MUZAFFARNAGAR

AFFILIATED TO

Dr A.P.J. ABDUL KALAM TECHNICAL UNIVERSITY,
UTTAR PRADESH, LUCKNOW

(2025-26)

DECLARATION

We, Naman Sharma (Roll No. 2406950100108) and Prerak (Roll No. 2406950100127), students of the Department of Computer Science and Engineering, in the third year of the Bachelor of Technology (B.Tech) program at Shri Ram Group of Colleges, hereby declare that the mini project titled "WanderLust" is the result of our work and research. The work presented in this report has been carried out under the supervision of Tanu Verma, Assistant Professor, Department of Computer Science and Engineering, Shri Ram Group of Colleges.

This report has not been submitted to any other university or institution for the award of any degree or diploma. The technologies and documentation used in developing the application are acknowledged in the references.

Date: _____

Place: _____

Signature: _____
Naman Sharma
Roll No. 2406950100108
B.Tech CSE, Third Year

Signature: _____
Prerak
Roll No. 2406950100127
B.Tech CSE, Third Year

Shri Ram Group of Colleges

Academic Session: 2025-26

CERTIFICATE

This is to certify that the mini project titled "WanderLust" has been carried out by Naman Sharma (Roll No. 2406950100108) and Prerak (Roll No. 2406950100127), students of the Department of Computer Science and Engineering, in the third year of the Bachelor of Technology (B.Tech) program at Shri Ram Group of Colleges. The project report is prepared as part of the academic requirements for the mini project course during the session 2025-26.

The work has been conducted under the guidance and supervision of Tanu Verma, Assistant Professor, Department of Computer Science and Engineering, Shri Ram Group of Colleges.

The project demonstrates the application of web development concepts to travel discovery and accommodation listing management. The implementation, verification evidence and current limitations are described in this report.

Students' signatures

_____

Naman Sharma
2406950100108

_____

Prerak
2406950100127
B.Tech CSE, Third Year

Supervisor's signature

_____

Tanu Verma
Assistant Professor
CSE Department
Shri Ram Group of Colleges

ACKNOWLEDGEMENTS

We are pleased to present the report of our B.Tech Mini Project "WanderLust", undertaken during our third year at Shri Ram Group of Colleges, Muzaffarnagar, in the academic session 2025-26.

We express our sincere gratitude to Tanu Verma, Assistant Professor, Department of Computer Science and Engineering, for her guidance and support throughout this project. Her suggestions helped us organize the work and improve the application and its documentation.

We thank the faculty members of the Department of Computer Science and Engineering for their assistance and cooperation. We also acknowledge Shri Ram Group of Colleges for the academic environment in which we could apply our knowledge of web application development.

We are grateful to our parents and friends for their encouragement. We acknowledge the official documentation and open-source technologies that supported the implementation and verification of WanderLust.

Naman Sharma - Roll No. 2406950100108

Prerak - Roll No. 2406950100127

B.Tech CSE, Third Year

Shri Ram Group of Colleges

ABSTRACT

WanderLust is a travel accommodation listing application designed to help visitors discover stays and help registered users manage property information. It brings titles, photographs, descriptions, destinations, nightly prices and reviews into a responsive web interface. The application uses Node.js and Express.js for request handling, MongoDB with Mongoose for storage, and EJS with Bootstrap for server-rendered presentation.

The implemented discovery features include literal search across title, location and country, inclusive price filters, price sorting and server-side pagination. Users can save stays in the current browser, open location-area map links and explore related listings. Review summaries use actual review documents. Existing authentication, listing management and review workflows remain part of the application.

Verification evidence includes eighteen unit tests, valid and invalid HTTP query cases, pagination checks, responsive layout checks and automated accessibility scans. Thirty-two fictional demonstration listings provide repeated destinations and varied prices for search verification. Their import uses stable identifiers and add-only behavior. Successful uploads, complete authenticated mutation workflows and live deployment behavior were not independently verified in the cloud environment. Booking, payments and account-synchronized saved stays remain future work.

Keywords: travel discovery, accommodation listings, Express.js, MongoDB, EJS, search, responsive interface.

TABLE OF CONTENTS

Declaration	i
Certificate	ii
Acknowledgements	iii
Abstract	iv
List Of Figures	vi
List Of Tables	vii
Chapter 1: Introduction	1
1.2 Objectives and scope	2
Chapter 2: Problem Statement	3
2.4 System requirements	4
2.6 Feasibility and setup	5
Technologies used	6
System architecture	7
Chapter 3: Implementation And Results	8
3.2 Database design	9
3.3 Search and filters	10
3.4 Pagination and saved stays	11
3.6 Reviews and related discovery	12
3.9 Authentication and access control	13
3.10 to 3.14 User interface screenshots	14
3.15 Demonstration data	19
3.16 Test strategy	20
3.17 Search and filter results	21
3.18 Discussion and limitations	22
Chapter 4: Conclusion And Future Work	23
4.2 Future work	24
References	25

LIST OF FIGURES

Figure 1 System architecture of WanderLust 7
Figure 2 Relationships among User Listing and Review 9
Figure 3 Current listings interface in local development 14
Figure 4 Manali search with inclusive price filters 15
Figure 5 Listing details in the local development version 16
Figure 6 Saved stays in the current browser 17
Figure 7 Login page in local development 18

LIST OF TABLES

Table 2.1 Functional requirements 4
Table 2.2 Feasibility assessment 5
Table T.1 Technology stack 6
Table 3.1 Implemented modules 8
Table 3.2 Main listing fields 9
Table 3.3 Supported discovery query parameters 10
Table 3.4 Intended access rules 13
Table 3.5 Demo listing distribution by nightly price 19
Table 3.6 Recorded verification evidence 20
Table 3.7 Search and filter verification cases 21

Chapter 1

INTRODUCTION

Travelers compare destination, nightly price and property information before choosing where to stay. When this information is scattered or difficult to search, finding suitable accommodation takes more effort. A travel discovery application can organize these details and make comparison easier.

WanderLust is a web application for browsing and managing accommodation listings. A listing contains a title, description, image, location, country and nightly price. Users can inspect property details, search destinations, narrow results by budget and read community reviews.

1.1 PROJECT BACKGROUND

The project applies full-stack development concepts through an existing Express and EJS architecture. The server handles browser requests, validates discovery parameters, queries MongoDB through Mongoose and renders HTML. Passport supports authentication, while the configured upload integration uses Multer and Cloudinary.

The project emphasizes working controls and truthful presentation. Saved stays are explicitly local to the current browser. Review averages are derived from stored reviews. Location links identify an area rather than an unverified street address.

The resulting application supports travel discovery and listing management. It does not currently process reservations, availability or payments. This distinction defines the scope of the implementation and the interpretation of its results.

OBJECTIVES AND SCOPE

1.2 PROJECT OBJECTIVES

- Provide a responsive interface for discovering and comparing accommodation.
- Connect title and destination search, price filters and sorting to MongoDB queries.
- Present results in manageable pages while retaining selected discovery values.
- Retain authentication, listing ownership and review workflows in the existing architecture.
- Support browser-local saved stays, actual review summaries and useful destination links.
- Document repeatable checks, demonstration data and unverified integrations honestly.

1.3 APPLICATION SCOPE

Visitors can browse, search and inspect listings without logging in. Registered users can access listing creation and contribute reviews. Listing owners have edit and delete controls for their properties. Saved stays are available through the current browser without introducing a new database relationship.

The project retains Node.js, Express.js, MongoDB, Mongoose, EJS and Bootstrap. No booking engine, payment workflow, category schema, availability calendar, messaging service or administrator dashboard is presented as implemented.

1.4 INTENDED USERS

The main users are visitors looking for stays, registered travelers contributing reviews and property owners maintaining their listings. The owner role is established by the existing user reference on each listing.

Chapter 2

PROBLEM STATEMENT

2.1 PROBLEM DEFINITION

The problem is to build a usable travel listing experience in which interface controls produce relevant database results and users can manage their contributions with appropriate access checks. Discovery must support both destination and budget constraints without requiring visitors to inspect every listing.

2.2 PRACTICAL CHALLENGES

• Combining a search term with minimum and maximum nightly prices.

• Rejecting malformed query values without passing request objects to MongoDB.

• Maintaining a predictable result order across multiple pages.

• Distinguishing device-local saved preferences from account-based persistence.

• Presenting missing images, no results and unavailable listings clearly.

• Testing with varied data while preserving existing records.

2.3 PROPOSED APPROACH

WanderLust separates query validation from controller actions and renders the selected values back into EJS controls. MongoDB handles matching, sorting and pagination. Reusable templates provide consistent listing cards and review summaries. Browser storage holds saved listing identifiers, while a read-only endpoint supplies their current public details.

A separate add-only demonstration dataset supports repeated locations, similar titles and exact price boundaries. Local checks compare results with expected records and document their coverage and limitations.

SYSTEM REQUIREMENTS

2.4 FUNCTIONAL REQUIREMENTS

Table 2.1 Functional requirements

ID	Requirement
F1	Browse stays and inspect listing details.
F2	Search title, location and country using one search term.
F3	Combine inclusive price limits with search and allowed sorting.
F4	Paginate results with selected values preserved.
F5	Register, log in and access existing protected workflows.
F6	Manage owned listings and contribute genuine reviews.
F7	Save and unsave listings on the current device.
F8	Display actual review summaries, related stays and map links.

2.5 NON-FUNCTIONAL REQUIREMENTS

The interface should remain readable and usable across mobile, tablet and desktop sizes. Server validation should reject invalid parameters and return useful errors. Shared templates and utilities should keep presentation and discovery logic maintainable.

Privacy requires credentials to remain outside public artifacts. Reliability requires meaningful behavior for unavailable images, missing listings and storage failures. Bounded result pages reduce per-response rendering, but no production performance benchmark is claimed.

FEASIBILITY AND DEVELOPMENT SETUP

2.6 FEASIBILITY ASSESSMENT

Table 2.2 Feasibility assessment

Aspect	Assessment
Technical	The existing JavaScript stack supports database queries, server rendering, sessions and upload integration.
Operational	Visitors use a browser; authenticated users access contribution and ownership controls.
Economic	Development can use existing tools and local MongoDB. Hosted services depend on provider plans and quotas.
Maintenance	Features remain within the current routes, controllers, utilities and EJS templates.

2.7 DEVELOPMENT ENVIRONMENT

The recorded development work used Node.js 22.19.0 and a local MongoDB service. Private environment configuration provides the database URI and session secret. New or replacement uploads additionally require valid Cloudinary configuration.

The existing dependency lockfile is retained. Installation uses npm ci --legacy-peer-deps because the current storage adapter and Cloudinary SDK declare different peer versions. The application starts with node app.js and normally listens on port 8080.

Development setup excludes the original destructive initializer. The separate add-only demo script runs only through an explicit command and is not part of normal application startup.

TECHNOLOGIES USED

Each technology has a defined role in request processing, persistence or presentation. EJS executes on the server; the browser receives the generated HTML.

Table T.1 Technology stack

Technology	Role in WanderLust
Node.js	Runtime that executes the server-side JavaScript.
Express.js	Routes, middleware, request handling and responses.
MongoDB / Atlas	Stores listing, user, review and session documents.
Mongoose	Models, schema definitions and database operations.
EJS / ejs-mate	Server-rendered HTML and reusable page layouts.
Bootstrap / CSS	Responsive controls and consistent interface styling.
Passport	Local authentication and session integration.
Multer / Cloudinary	Configured multipart upload and image-storage flow.
Joi	Discovery, listing and review input validation.
Browser localStorage	Device-local saved listing identifiers.

Client JavaScript handles save controls, photo fallbacks, form feedback and other progressive interactions. Core search, price filtering, sorting and pagination are performed through server-side database queries.

SYSTEM ARCHITECTURE

The application follows a layered request flow within one Express project. Routes select an action, middleware checks access or input, controllers use Mongoose models and EJS templates render the response.

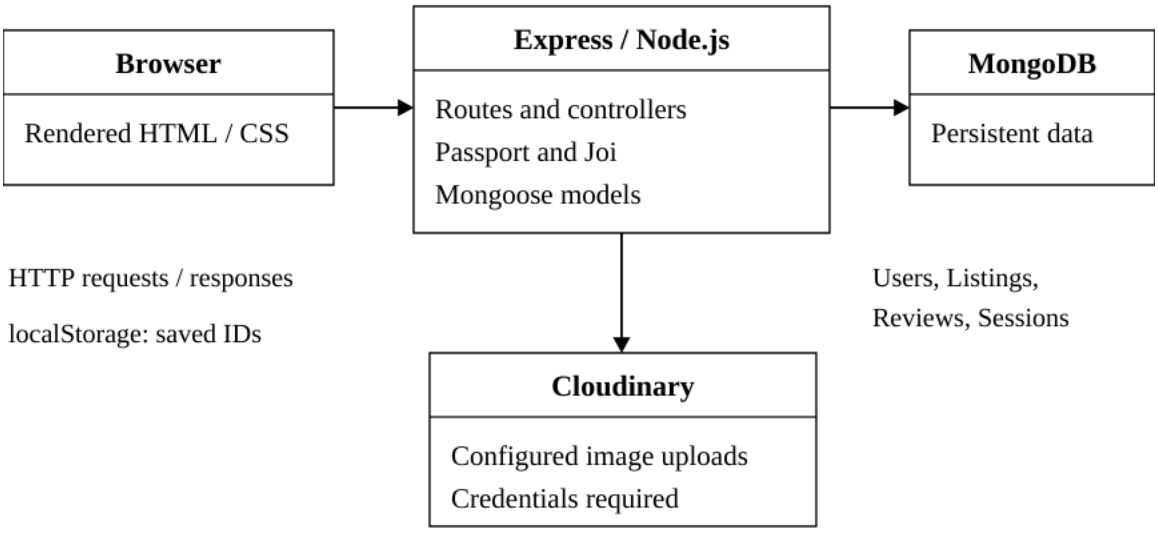


Figure 1 System architecture of WanderLust

REQUEST AND RESPONSE FLOW

A GET request to /listings carries the search term and optional price, sort and page parameters. Validated values become explicit MongoDB conditions. The controller counts matches, retrieves the requested page and supplies both results and selected values to the template.

The saved-stays interface also uses a bounded JSON endpoint for current public listing details. Saved identifiers remain in localStorage. Cloudinary is a configured image-upload integration; ordinary external map links require no embedded paid mapping API.

Chapter 3

IMPLEMENTATION AND RESULTS

3.1 MODULE ORGANIZATION

Table 3.1 Implemented modules

Module	Responsibilities
Discovery	Validate search and price values; query, sort and paginate.
Listings	Display details and retain existing management actions.
Authentication	Register users, authenticate and maintain sessions.
Reviews	Retain review workflows and calculate real summaries.
Saved stays	Persist browser-local IDs and retrieve public details.
Related discovery	Select other stays and construct area map links.
Presentation	Render shared cards, forms, empty states and feedback.
Demo and checks	Provide add-only fixtures and repeatable verification.

The implementation keeps the current Express, EJS and Mongoose architecture. Discovery helpers separate parameter validation, pagination and review arithmetic from the controller. Shared card and review partials keep the listing, related and saved interfaces consistent.

The model relationships remain unchanged. Saved preferences introduce no account field, and demo records use only supported schema fields. This report describes the current implementation and separates local verification from production behavior.

DATABASE DESIGN

3.2 DATA ENTITIES AND RELATIONSHIPS

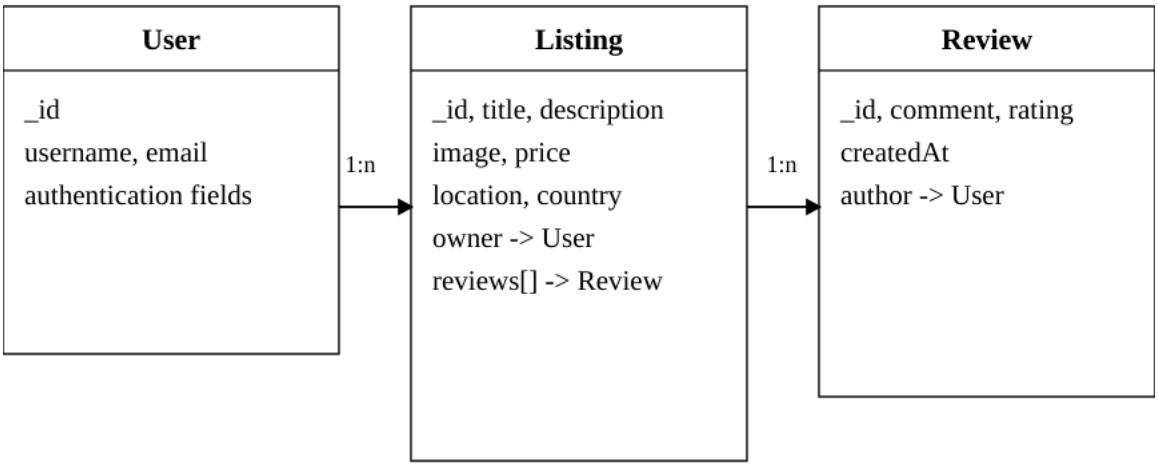


Figure 2 Relationships among User Listing and Review

Table 3.2 Main listing fields

Field	Type and purpose
title	Required String identifying the stay.
description	String describing the accommodation.
image.url / filename	Strings storing the image reference.
price	Number representing INR per night.
location / country	Strings identifying the destination.
owner	ObjectId reference to User.
reviews	Array of ObjectId references to Review.

User records support the existing Passport authentication plugin. Review records contain comment, rating, author and creation date. Saved stays use browser storage rather than a new schema field. Fixture identifiers use the existing _id property.

SEARCH AND FILTER IMPLEMENTATION

3.3 DISCOVERY PARAMETERS

Table 3.3 Supported discovery query parameters

Parameter	Behavior
q	Trimmed literal search across title, location and country; maximum 100 characters.
minPrice	Inclusive lower nightly price; non-negative.
maxPrice	Inclusive upper nightly price; non-negative.
sort	Allowlisted default, price_asc or price_desc.
page	Positive whole number up to 1,000,000.

3.3.1 Validation and query construction

The server rejects malformed or repeated scalar values, unsupported sorts and reversed price ranges. Price limits cannot exceed 100,000,000. Regex characters in search text are escaped, so a term is interpreted literally. Only explicit predicates are passed to MongoDB.

Search uses a case-insensitive OR across title, location and country. Price constraints use inclusive comparisons and combine with that search condition. Country participates in the search; a separate country or category filter is not implemented.

3.3.2 Combined example

The query /listings?q=Manali&minPrice=1000&maxPrice=3000 matches two demo listings priced INR 1,200 and INR 2,800. Existing records may add further matches. Submitted values remain visible, and clearing filters returns to the normal listing page.

PAGINATION AND SAVED STAYS

3.4 SERVER SIDE PAGINATION

The controller uses countDocuments to obtain the match count, then applies sort, skip and limit to fetch at most twelve listings. Default ordering uses _id, and price sorting uses _id to break ties. Stable ordering prevents the tested pages from repeating or omitting records.

Pagination links retain search, price and sorting values. A new search resets the page. Invalid pages return an error; pages beyond the result range redirect to the last available page, or page one when no results exist.

3.5 SAVED ON THIS DEVICE

Save controls store up to one hundred valid listing IDs in the current browser. The saved page retrieves current public details through /listings/saved/data. The endpoint validates identifiers, bounds the query and preserves the requested order.

Saving survives reloads and synchronizes between tabs of the same browser and origin. It does not synchronize across devices or accounts. Clearing browser storage removes the saved collection.

A failed request retains saved IDs and offers a retry. Confirmed missing listings are removed only after a successful response. Blocked storage and failed writes are explained without reporting a false saved state.

REVIEWS AND RELATED DISCOVERY

3.6 ACTUAL REVIEW SUMMARIES

Listing cards and details derive review counts and averages from actual review documents. Only finite numeric ratings from one to five contribute to the average. Missing references are ignored. A listing without reviews displays "No reviews yet" rather than an invented score.

The count of review documents and the count of valid rated reviews are distinct. These values are calculated on reads rather than stored as new listing fields. The existing review creation, deletion and author controls remain part of the project.

3.7 RELATED STAYS

A detail page can display up to four other listings. Bounded MongoDB queries prioritize the same stored location and then fill remaining places from the same country. The current listing and already selected IDs are excluded. No related section is shown when suitable records do not exist.

3.8 LOCATION LINKS

The map link encodes the existing location and country into a Google Maps search URL. It identifies a destination area and does not claim an exact property address. No mapping key or paid API integration is required.

Related stays and maps improve discovery without implying a booking service. They help users explore additional options and understand the destination context.

AUTHENTICATION AND ACCESS CONTROL

3.9 EXISTING AUTHENTICATION WORKFLOW

Passport local authentication handles login, while the existing user plugin supports account registration and password hashing. Express sessions use the configured MongoDB session store. Logout removes the authenticated user state.

Table 3.4 Intended access rules

Action	Access rule
Browse and search	Public.
View details and saved page	Public; saved preferences remain local.
Create listing	Authenticated user.
Edit or delete listing	Authenticated owner.
Add review	Authenticated user with valid review input.
Delete review	Existing review-author check.

3.9.1 Evidence and boundaries

Read-only checks verified authentication redirects and create/edit page rendering using a temporary in-memory session for an existing local owner. Successful password login, uploads and complete mutation flows were not exercised in that environment.

The report documents existing controls and their intended behavior; it does not claim a comprehensive authorization or security audit. Successful submissions and additional negative-access scenarios remain integration checks before a release.

SCREENSHOTS OF USER INTERFACE

3.10 HOME AND LISTING DISCOVERY

The listings page combines a travel-focused hero with search controls, clear navigation and accommodation cards. Each card presents a title, location and nightly price in a consistent layout.

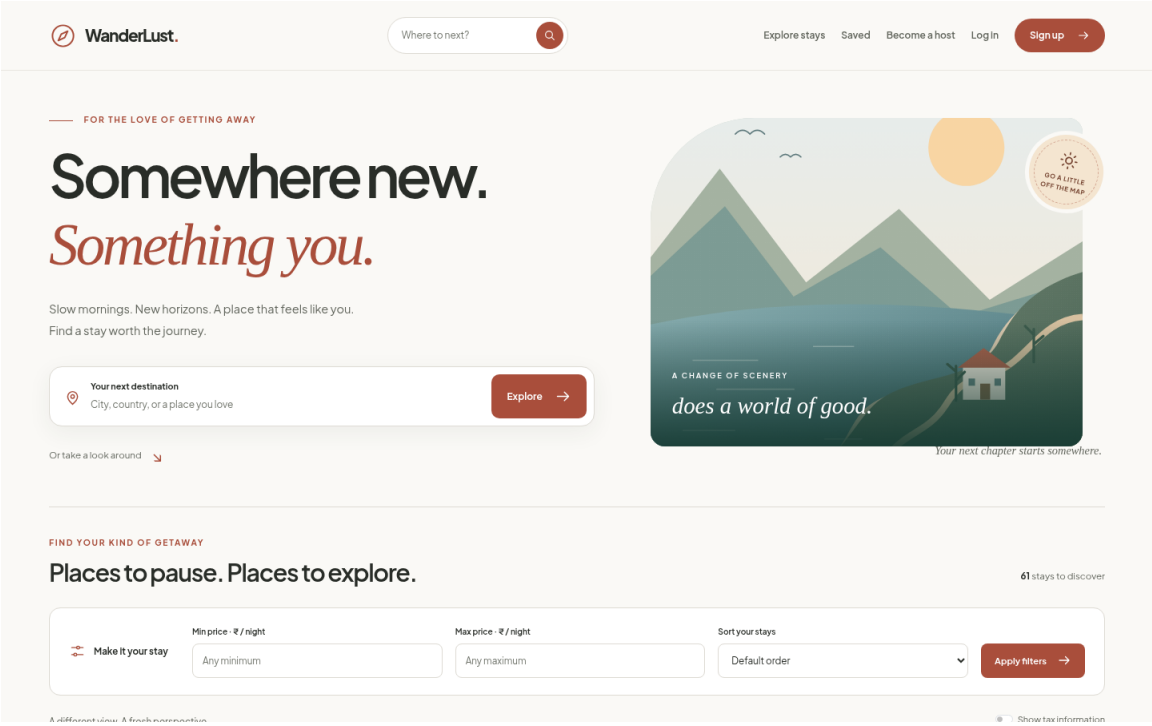


Figure 3 Current listings interface in local development

The warm brand accent, typography and shared footer create a consistent visual identity. Search and save controls correspond to implemented actions. Cards use responsive image proportions and truthful review summaries.

This is a local development capture, not independent evidence of the live deployment. External photo requests are restricted in the cloud environment, so the application may display its labelled fallback while retaining the stored image URLs.

SEARCH RESULTS INTERFACE

3.11 COMBINED DESTINATION AND PRICE SEARCH

The results interface retains the submitted search and price values. Visitors can select price order, inspect matching cards, clear filters or move through the paginated result set.

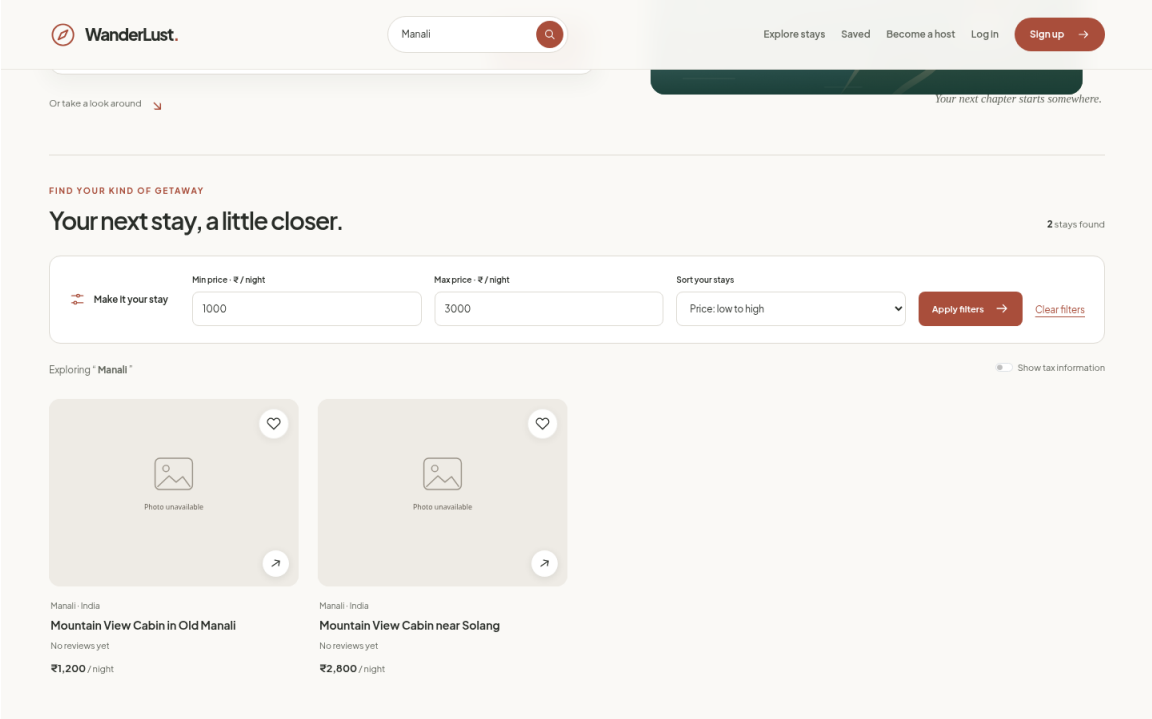


Figure 4 Manali search with inclusive price filters

The example searches Manali with INR 1,000 to INR 3,000 as the nightly budget. Two demo records match: INR 1,200 and INR 2,800. Search and filters execute in MongoDB rather than hiding unmatched cards only in the browser.

A useful no-results state explains when no stay matches the submitted criteria. Clearing filters returns to normal discovery. New search submissions begin on the first page, and pagination links preserve the selected values.

LISTING DETAILS INTERFACE

3.12 PROPERTY INFORMATION AND REVIEWS

The detail interface brings the main image, description, destination, nightly price and owner information together. Actual reviews and their summary appear without fabricated scores or testimonials.

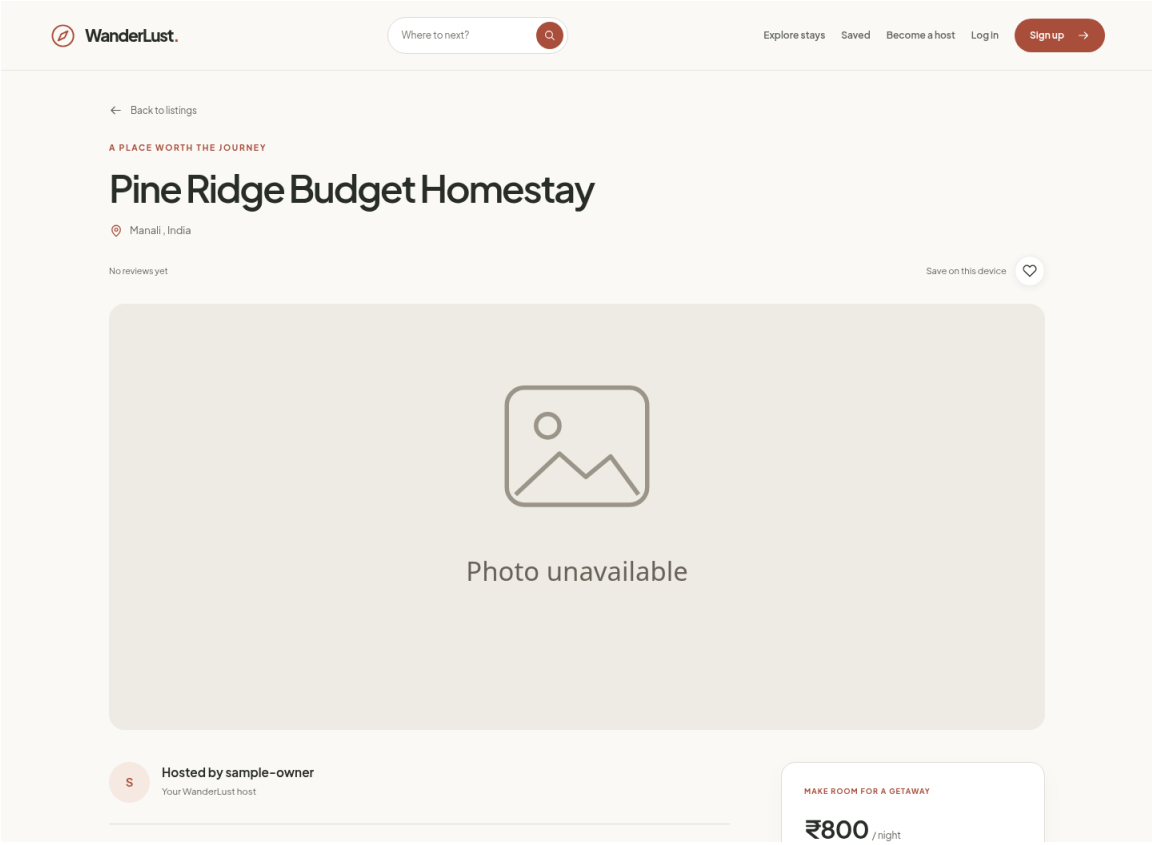


Figure 5 Listing details in the local development version

Visitors can save a listing on this device, open its location area in maps and explore related stays. Edit and delete controls are tied to the existing ownership workflow. Missing or invalid detail identifiers receive a useful 404 response.

The screenshot documents presentation, not a successful booking or payment. External image failures display the labelled fallback without overwriting the stored image reference.

SAVED STAYS INTERFACE

3.13 DEVICE LOCAL COLLECTION

The saved page clearly identifies its collection as "Saved on this device". It displays current details for stored IDs and provides working unsave actions.

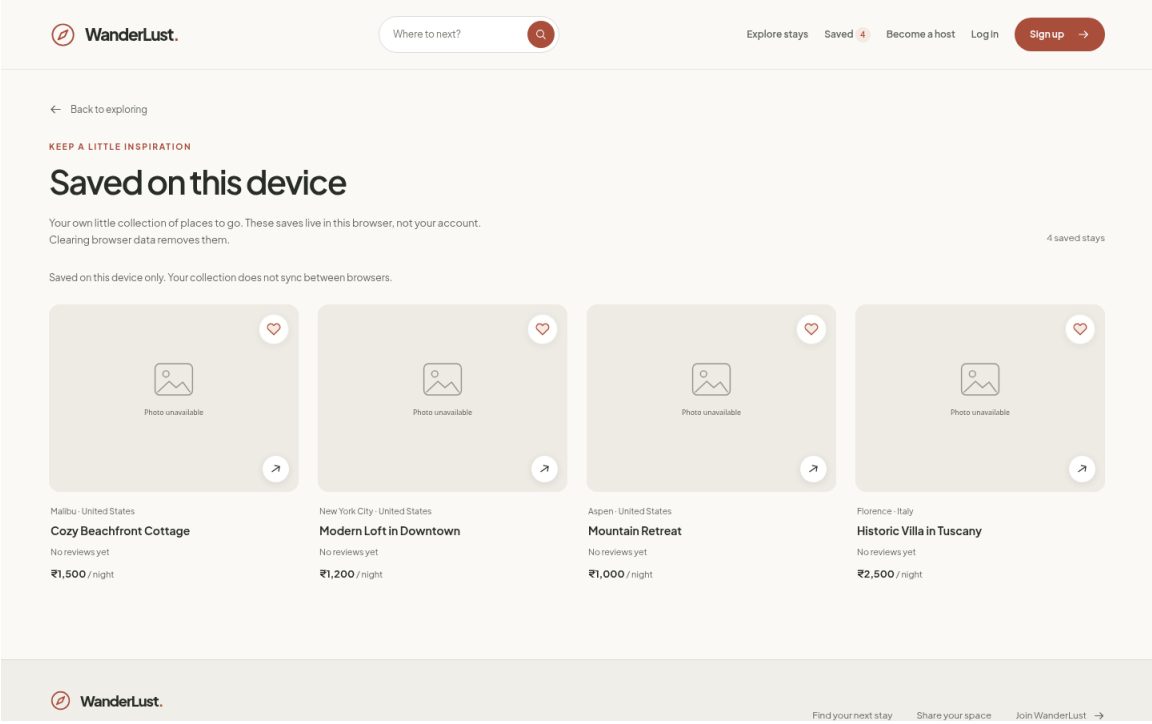


Figure 6 Saved stays in the current browser

The implementation distinguishes an empty collection from unavailable data. A request failure keeps the saved IDs and permits retrying. Confirmed deleted records are handled after a successful response. Browser storage errors do not silently create a saved state.

The collection is a convenience feature for the current browser. It does not create a user-to-listing relationship in MongoDB and does not imply account-based synchronization.

AUTHENTICATION AND FORM INTERFACE

3.14 SHARED FORM PRESENTATION

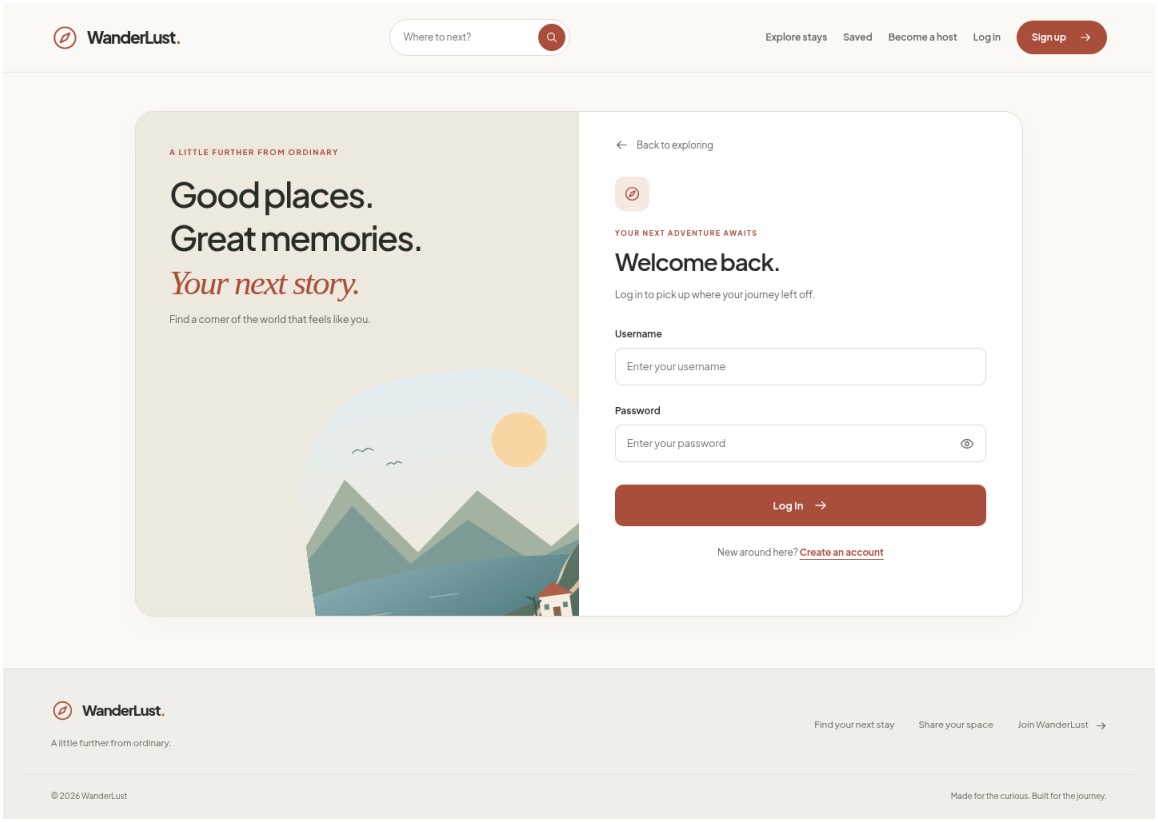


Figure 7 Login page in local development

Login, signup and listing forms use consistent labels, spacing and focus styles. Password visibility, invalid-field feedback and native form actions remain functional. The registration URL retains the existing spelling /singUp.

Valid submission attempts show "Please wait..." and block accidental duplicate submit events. Cancelled attempts and browser-history returns restore controls. Client feedback improves usability but does not replace server validation.

Listing forms collect the supported title, description, image, price, location and country fields. A selected image can be previewed locally; edits can preserve the current image. Actual new or replacement uploads require valid Cloudinary credentials and were not verified here.

DEMONSTRATION DATA

3.15 DATASET DESIGN

The separate fixture manifest contains thirty-two fictional accommodation listings across twenty destination strings. Prices range from INR 800 to INR 25,000 per night. Repeated locations, similar title phrases and varied budgets support individual and combined discovery checks.

Table 3.5 Demo listing distribution by nightly price

Price band in INR	Demo listings
800 to 1,500	5
1,501 to 5,000	15
5,001 to 10,000	8
Above 10,000 up to 25,000	4
Total	32

Destinations include Manali, Shimla, Mussoorie, Nainital, Goa, Jaipur, Udaipur, Jaisalmer, Rishikesh, Varanasi, Darjeeling, Munnar, Mumbai, Delhi, Bengaluru and Hyderabad, plus Ubud in Bali, Kyoto, Interlaken and Lisbon.

3.15.1 ADD ONLY IMPORT

Stable _id values make repeated imports idempotent. Preview performs no writes; explicit insertion creates only missing records and skips existing IDs. The script does not update, delete or reset records. The supplied import log records thirty-two insertions into the selected live demo database using an existing owner account. This result was not independently verified against the live service.

TEST STRATEGY

3.16 LEVELS OF VERIFICATION

Verification combines isolated logic checks, read-only HTTP requests and browser observations. Each level answers a different question: whether validation is correct, routes return the expected results and the interface remains usable at tested screen widths.

Table 3.6 Recorded verification evidence

Check group	Recorded result
Logic and seed safety	18 unit tests passed: 12 discovery/query tests plus 6 seed tests.
HTTP query cases	17 valid and 10 invalid cases passed.
Pagination	15 exact-ID pages checked without gaps or duplicates.
Responsive layout	45 checks across 9 states and 5 widths.
Accessibility	20 scans; zero automated violations detected.

Responsive widths were 320, 375, 768, 1024 and 1440 pixels. Recorded browser checks also exercised saving, retry behavior, keyboard activation, map links and submission feedback without completing successful mutation requests.

The local HTTP harness guarded model writes, used in-memory test sessions and compared listing hashes before and after. Its sixty-one local records were unchanged. Automated accessibility scans leave some contrast checks for manual review and do not certify complete accessibility.

SEARCH AND FILTER TEST RESULTS

3.17 REPRESENTATIVE EXPECTATIONS

Table 3.7 Search and filter verification cases

Scenario	Expected behavior or demo contribution
Complete title	Mountain View Cabin in Old Manali matches 1 demo stay.
Partial title	Mountain View Cabin matches 2 demo stays.
Location	Manali matches 4; Goa matches 3 demo stays.
No matching term	A unique nonexistent term gives an empty result.
Minimum price	Only records at or above the lower bound appear.
Maximum price	Only records at or below the upper bound appear.
Combined filters	Manali, INR 1,000 to 3,000: 2 demo stays.
Inclusive boundaries	Exact INR 800 and INR 25,000 each match 1 demo stay.
Sorting and clearing	Prices follow selected order; clearing restores discovery.

Demo counts describe only the thirty-two fixtures. Other records may add matches, so the total production count should not be assumed from this table. Local HTTP checks verified the existing queries against the populated local database; they did not certify the current live data.

Malformed scalar values, unsupported sort options and reversed ranges return validation errors. Empty or absent search parameters behave like normal listing discovery, with pagination available for all matches.

RESULT DISCUSSION AND LIMITATIONS

3.18 OBSERVED OUTCOMES

The implementation connects discovery controls to real database queries, retains selected values and bounds result pages. Device-local saves, real review summaries, location links and related stays improve discovery without changing the existing model relationships.

Recorded local checks support the tested query, pagination and interface behavior. The add-only fixtures provide varied search cases, while stable identifiers prevent duplicate imports. The supplied live-import log remains separate from the independently observed local checks.

3.19 CURRENT LIMITATIONS

- Successful real-password login, complete create/edit/delete submissions, review mutations and actual Cloudinary uploads were not verified in the cloud.
- External photo hosts are restricted in this environment; current remote image availability is not guaranteed.
- GitHub changes were pushed, but Render deployment and live behavior were not independently verified.
- Saved stays do not synchronize across accounts or devices.
- The local verification dataset contains no real reviews; nonzero rating arithmetic uses isolated test fixtures.
- No production load test, complete accessibility assessment or comprehensive security audit was performed.

These limits define the evidence behind the reported outcomes. Unverified integrations and proposed features are not presented as completed test results.

Chapter 4

CONCLUSION AND FUTURE WORK

4.1 CONCLUSION

WanderLust demonstrates the integration of server-side JavaScript, a document database and dynamic templates in a travel discovery application. Visitors can browse stays, search destinations, compare prices, inspect details and save listings in the current browser. Existing authenticated workflows support property and review contributions.

The implementation keeps the Express, Mongoose, EJS and Bootstrap architecture. Reusable templates provide a consistent interface, while validated discovery parameters and explicit database queries support predictable search, sorting and pagination. Real review summaries avoid fabricated social proof.

The fictional demonstration dataset supplies repeated destinations, similar titles and prices from INR 800 to INR 25,000. Its add-only import preserves existing records and skips stable identifiers on subsequent runs. A supplied execution log records a completed import into the selected live database; independent live verification remains outstanding.

Recorded verification includes eighteen unit tests, twenty-seven HTTP query cases, fifteen pagination pages, forty-five responsive checks and twenty automated accessibility scans. These results support the tested local behavior while leaving successful upload, authentication mutation, release and manual accessibility checks clearly identified.

The project provides practical experience with request handling, model relationships, authentication, responsive presentation and repeatable verification. It is currently a discovery and listing management system, with booking-related services reserved for future scope.

FUTURE WORK

4.2 PRIORITIZED EXTENSIONS

- Verify successful authentication, listing and review submissions in an isolated test database, including negative-access scenarios.
- Validate real upload behavior with approved credentials and resolve storage-adapter compatibility concerns.
- Add account-synchronized saved stays only after approving a persistence design and access rules.
- Design availability and booking lifecycles, including overlapping-date prevention, before implementing reservations.
- Integrate payments only after defining booking states, verification, failure recovery and refund behavior.
- Evaluate indexes and query performance using representative record volumes and measured response times.
- Extend manual keyboard, contrast and assistive-technology testing beyond automated scans.

4.3 RELEASE VERIFICATION

Before claiming a live update, verify the existing Render service, its environment configuration and the deployed application. Test discovery, details, authentication and contribution workflows using approved test records. A successful Git push alone does not establish a successful deployment.

Categories require an approved schema addition because the current Listing model has no category field. Booking, payments, availability and account-based saved collections remain proposals rather than implemented capabilities.

REFERENCES

[1] WanderLust original project repository.
https://github.com/Namansharma644/WanderLust

[2] WanderLust existing public listings address.
https://wanderlusts-kjty.onrender.com/listings

[3] Node.js documentation.
https://nodejs.org/docs/latest-v22.x/api/

[4] Express routing documentation.
https://expressjs.com/en/guide/routing.html

[5] MongoDB Atlas documentation.
https://www.mongodb.com/docs/atlas/

[6] Mongoose schema documentation.
https://mongoosejs.com/docs/guide.html

[7] EJS template engine documentation.
https://ejs.co/

[8] Bootstrap documentation.
https://getbootstrap.com/docs/5.3/

[9] Passport authentication documentation.
https://www.passportjs.org/

[10] Cloudinary documentation.
https://cloudinary.com/documentation/

[11] Joi validation documentation.
https://joi.dev/api/

[12] Institutional format supplied for this report.
Mini Project report third year Format.pdf